Document number	P3810R1
Date	2025-10-03
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	SG23 Safety and Security

hardened memory safety guarantees

Table of contents

Changelog

R1

- fixed copy paste errors

Abstract

The `C++` standard could say that all of its hardening is itself free of memory safety related undefined behavior if it required that STL implementations make the following functions free of memory safety related undefined behavior: `size()`, `empty()`, iterator difference, `static_extent()`, `extent()`, `has_value()`, `valueless_by_exception()`, `holds_alternative<I>()`.

Following is hopefully a unique list of the hardened contract assertions currently proposed for the `C++26` standard.

Standard Library Hardening [1]

```
idx < size() is true
empty() is false
Count <= size() is true
count <= size() is true
Offset <= size() && (Count == dynamic_extent || Count <= size() - Offset) is true
offset <= size() && (count == dynamic_extent || count <= size() - offset) is true
count == extent is true
(last - first) == extent is true
ranges::size(r) == extent is true
il.size() == extent is true
s.size() == extent is true
pos < size() is true
empty() is false
n <= size() is true
a.empty() is false
n < a.size() is true
static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r) is true
n < size() is true
has_value() is true
has_value() is false
```

Minor additions to C++26 standard library hardening [2]

```
i >= 0
i < N
!empty() is true
length > 0 is true
n < length is true
i.length > 0 is true
x.length > 0 and y.length > 0 are true
n >= 0
n <= length is true
-n <= length is true
x.v_.valueless_by_exception() is false
holds_alternative<I>(v_) is true
x.v_.valueless_by_exception() and y.v_.valueless_by_exception() are each false
skip <= skip + max_depth is true
frame_no < size()
```

Is it unreasonable to require a trivial to write member function to be written free of memory safety related undefined behavior? Consider for example the two most common ways to write a size member function.

```
class something
{
private:
    size_t s;
public:
    constexpr size_t size() const
    {
        return s;
    }
};
```

```
class something
{
private:
    iterator start;
    iterator end;
public:
    constexpr size_t size() const
    {
        return end - start;
    }
};
```

Both of these common implementations are free of uninitialized, range access, null pointer dereference and use after free memory errors. Neither is their type safety errors. So requiring something that STL implementors are already doing does not seem unreasonable. It is similar for the other value semantic functions that simply return copies of members of the class in question.

Motivation

There has been sustained interest in contracts being free of undefined behavior. This was a part of the `Contracts - Use Cases` [3] [4] back in 2020. There has been a lot of interest since.

2022 Contracts for C++: Prioritizing Safety [5] 2024 P2900R6 May Be Minimal, but It Is Not Viable [6] 2024 Contracts: Protecting The Protector [7] 2024 Static analysis and 'safety' of Contracts, P2900 vs. P2680/P3285 [8] 2025 Contract concerns [9]

Most every day programmers are more specifically concerned about the memory unsafety subset of undefined behavior. While we can't at this moment mandate that contracts are free of memory unsafety, can we at least offer the guarantee that the standard by default won't deliberately create hardened contracts with memory safety related undefined behavior without a very good reason for doing so.

Impact on the standard

Safety not only requires safety to be in the language but also a part of the libraries. This proposal provides safety guarantees in some portions of the standard by requiring STL implementers to make some of their functions safe.

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3471r4.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3471r4.html) ↩
2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3697r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3697r0.html) ↩
3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1995r1.html#crit.noundef (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1995r1.html#crit.noundef) ↩
4. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1995r1.html#crit.noassume (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1995r1.html#crit.noassume) ↩
5. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2680r1.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2680r1.pdf) ↩
6. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3173r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3173r0.pdf) ↩
7. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3285r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3285r0.pdf) ↩

8. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3362r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3362r0.html) ↩
9. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3573r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3573r0.pdf) ↩